오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

JavaScript 기초

DAY 14

배열 메서드와 데이터 처리

함수를 넘겨 배열을 한 번에 바꾸고 · 고르고 · 모으는 메서드로 할 일 목록과 장바구니를 가공합니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **함수를 넘겨 배열을 한 번에 다룰 수 있다**

   forEach로 하나씩 처리하고, map으로 바꾸고, filter로 고릅니다.

2. **reduce · find · some · every를 골라 쓸 수 있다**

   합계를 내고, 하나를 찾고, 조건에 맞는지 true · false로 확인합니다.

3. **원래 배열이 바뀌는지 구분할 수 있다**

   새 배열을 돌려주는 메서드와 원래 배열을 바꾸는 메서드를 나눕니다.

오늘의 결과물

할 일 목록을 다루는 todos.js와 장바구니를 계산하는 cart.js

ROADMAP

오늘의 흐름

01

콜백과 forEach

함수를 넘기는 모양
for와 비교

02

map · filter

바꾸기와 고르기
실습 01

실습 포함

03

reduce와 찾기

합계 · find
some · every

04

원본과 데이터 가공

원래 배열이 바뀌는가
실습 02

실습 포함

앞의 두 섹션은 하나씩 처리하고 · 바꾸고 · 고르는 메서드, 셋째 섹션은 모으고 · 찾는 메서드입니다.
마지막에는 원래 배열이 바뀌는지 따져 보고, 상품 할인과 장바구니 총액을 계산합니다.

되짚기 · 01

어제 배운 것 — for + if + push로 고르기

```
// DAY 13 42장 — 품절 상품의 이름 모으기
const soldOut = [];
for (let i = 0; i < products.length; i++) {
  if (products[i].stock === 0) {
    soldOut.push(products[i].name);
  }
}

// 오늘 배울 모양
const names = products.filter((p) => p.stock === 0).map((p) => p.name);
```

세 단계를 직접

빈 배열 만들기,
돌면서 검사하기,
push로 넣기.

오늘은 메서드에게

고르는 일은 filter,
꺼내는 일은 map이
대신합니다.

화살표 함수 — 한 줄이면 { }와 return 생략

```
// DAY 12 — 두 함수는 같은 일을 합니다
const double1 = (n) => {
  return n * 2;
};
const double2 = (n) => n * 2;      // { }와 return을 뺀 모양

console.log(double1(5));   // 10
console.log(double2(5));   // 10
```

{ }를 쓰면 return

본문을 { }로 감싸면
return으로 값을
돌려줘야 합니다.

한 줄이면 생략

=> 뒤의 식이 그대로
돌려줄 값이 됩니다.
오늘 가장 많이 씁니다.

준비

오늘의 셋팅

```
Day14/                    ← 같은 저장소 안에 만듭니다
  todos.html              ← todos.js 를 연결합니다 (실습 01)
  todos.js
  cart.html               ← cart.js 를 연결합니다 (실습 02)
  cart.js
$ git status
/* 오늘도 main에 바로 커밋합니다 */
```

짧은 예제는 Console

슬라이드의 한두 줄은
Console에 쳐서
바로 확인합니다.

실습은 파일로

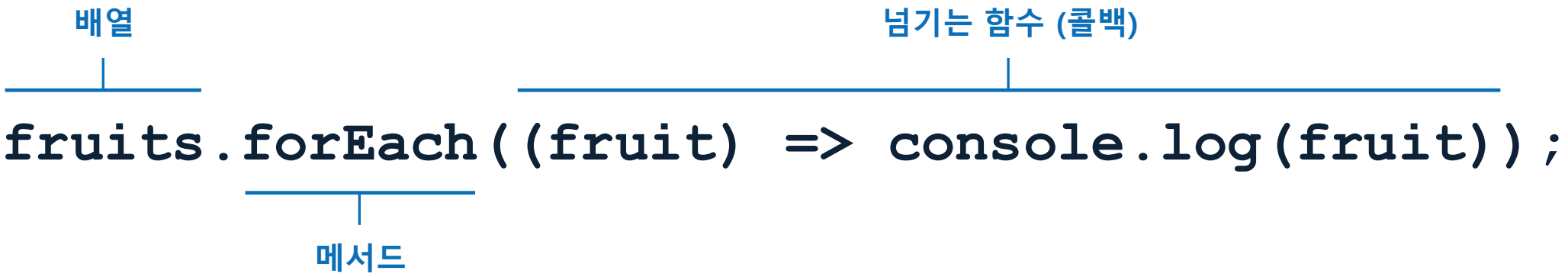
할 일 목록과 장바구니는
.js 파일에 쓰고
새로고침해 확인합니다.

01

콜백과 forEach

함수를 넘기는 메서드 · forEach · 번호 함께 받기 · for와 비교 · 멈출 수 없는 forEach

메서드에 넘기는 '값 하나마다 할 일'



forEach가 배열의 값을 하나씩 꺼내 넘긴 함수를 불러 줍니다

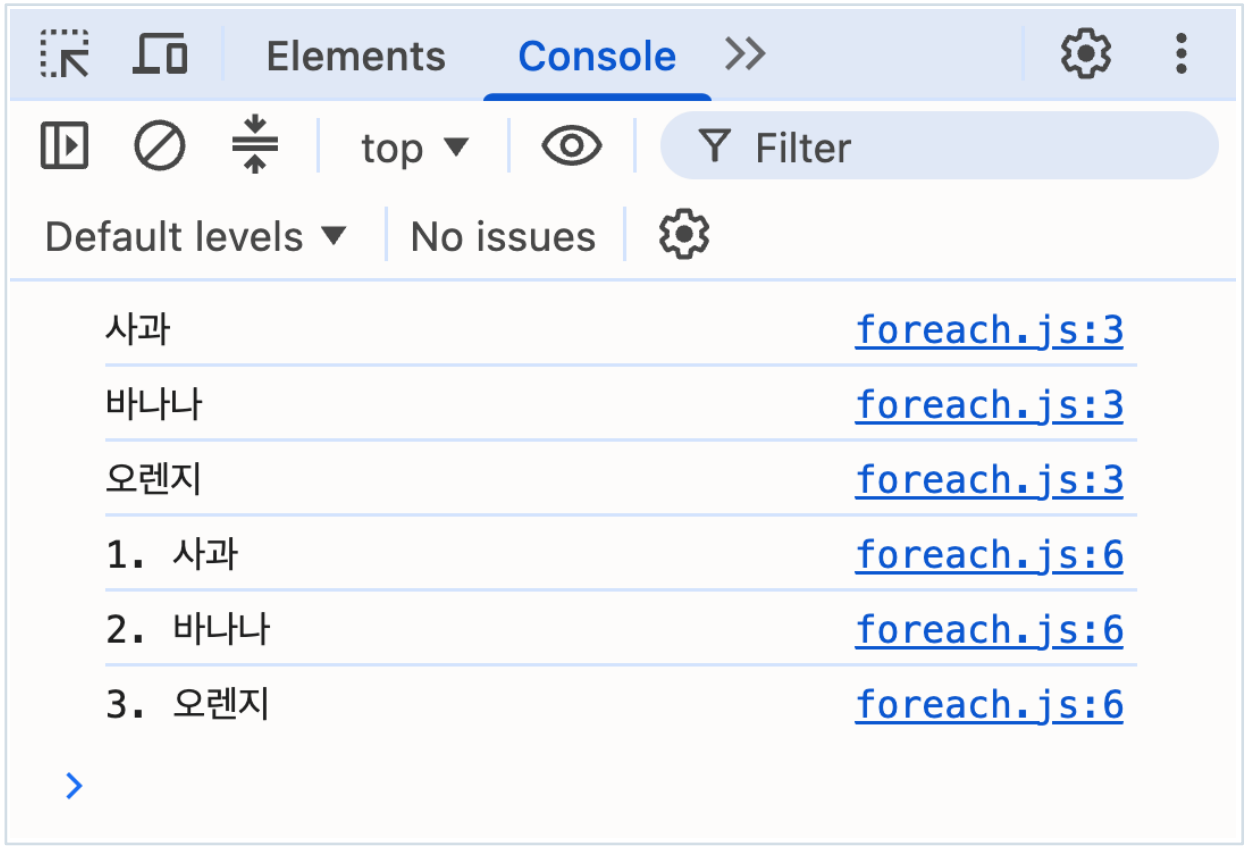
(fruit) 자리에 "사과", "바나나", "오렌지"가 차례로 들어갑니다.

값을 하나씩 꺼내 함수에 넣는 forEach

```
const fruits = ["사과", "바나나", "오렌지"];
fruits.forEach((fruit) => {
  console.log(fruit);
});
fruits.forEach((fruit, i) => {     // 번호도
  console.log(`${i + 1}. ${fruit}`);
});
```

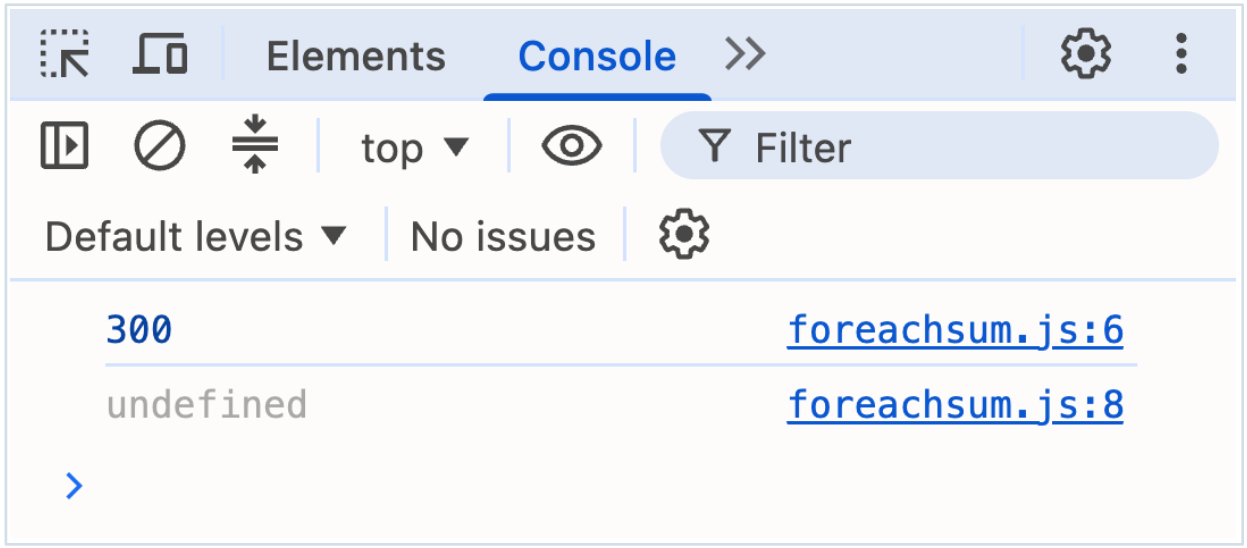
읽는 법

첫째 매개변수 — 꺼낸 값
둘째 매개변수 — 그 값의 번호
값이 셋이면 세 번 불립니다



합계는 바깥 변수에, 돌려주는 값은 없음

```
const scores = [80, 95, 70, 55];
let total = 0;                      // 바깥 변수
scores.forEach((score) => {
  total += score;
});
console.log(total);
const result = scores.forEach((score) => score * 2);
console.log(result);
```

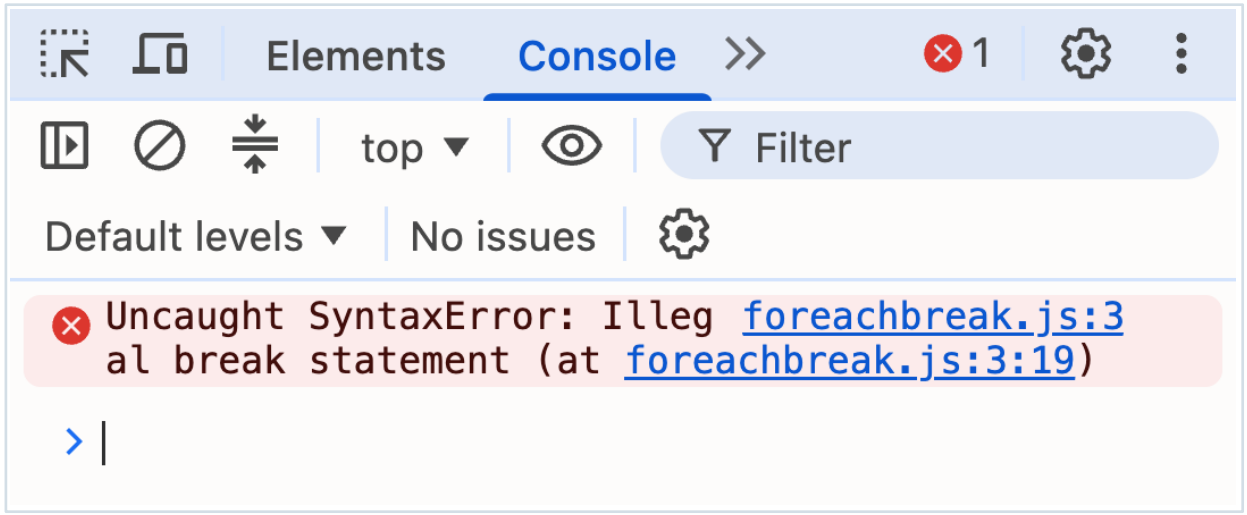


읽는 법

합계는 바깥의 total에 쌓입니다
forEach는 undefined를 돌려줍니다
결과를 변수에 담지 않습니다

break로 멈출 수 없는 forEach

```
const scores = [80, 95, 70, 55];
scores.forEach((score) => {
  if (score < 60) break;      // 멈추고 싶지만
  console.log(score);
});
```



멈춰야 한다면

for 문의 break를 씁니다
찾기 · 확인은 find · some · every
(섹션 03) — 답이 나오면 멈춥니다

01 · 비교

for와 forEach — 같은 일, 다른 모양

	for	forEach
모양	`for (let i = 0; i < arr.length; i++)`	`arr.forEach((value) => { … })`
값 꺼내기	`arr[i]`	매개변수로 바로 받기
번호	직접 만들고 늘리기	둘째 매개변수(i)
중간에 멈추기	`break`	할 수 없음

값마다 같은 일을 할 때는 forEach가 짧고, 멈춰야 하면 for를 씁니다

여기까지 정리

1	**콜백**	메서드에 넘기는 함수 — 값 하나마다 메서드가 불러 줍니다
2	**forEach**	값을 하나씩 꺼내 함수에 넣기 — (값, 번호) 순서로 받습니다
3	**돌려주는 값**	forEach는 늘 undefined — 결과를 변수에 담지 않습니다
4	**멈추기**	forEach는 break로 멈출 수 없음 — 멈춰야 하면 for

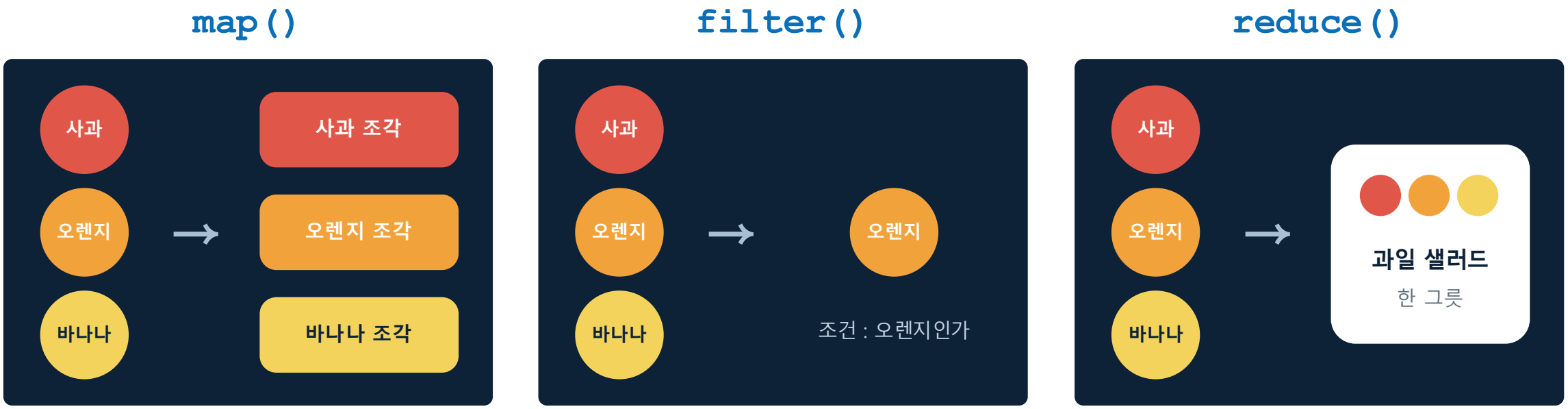
다음은 결과를 새 배열로 돌려주는 map과 filter입니다

02

map과 filter

한눈에 보기 · map으로 바꾸기 · return 빠뜨리기 · filter로 고르기 · 이어 쓰기 · 실습 01

바꾸기는 map, 고르기는 filter, 모으기는 reduce



map은 개수 그대로 바꾸고, filter는 맞는 것만 남기고, reduce는 하나로 모읍니다

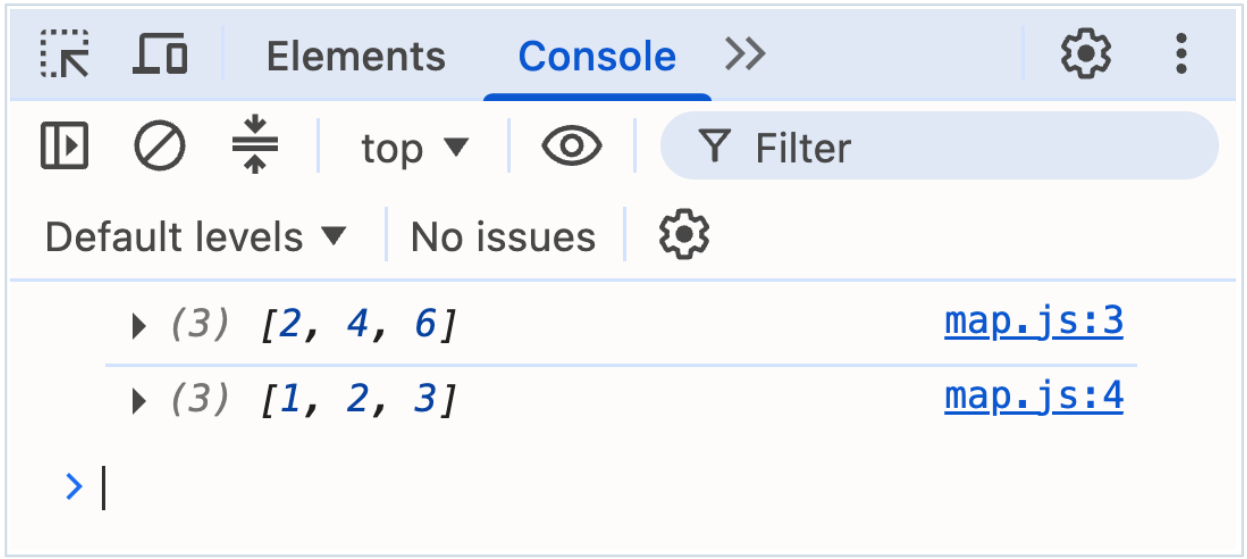
세 메서드 모두 원래 배열은 그대로 두고 새 값을 돌려줍니다.

하나하나 바꾼 새 배열을 돌려주는 map

```
const numbers = [1, 2, 3];
const doubled = numbers.map((num) => num * 2);
console.log(doubled);
console.log(numbers);        // 그대로
```

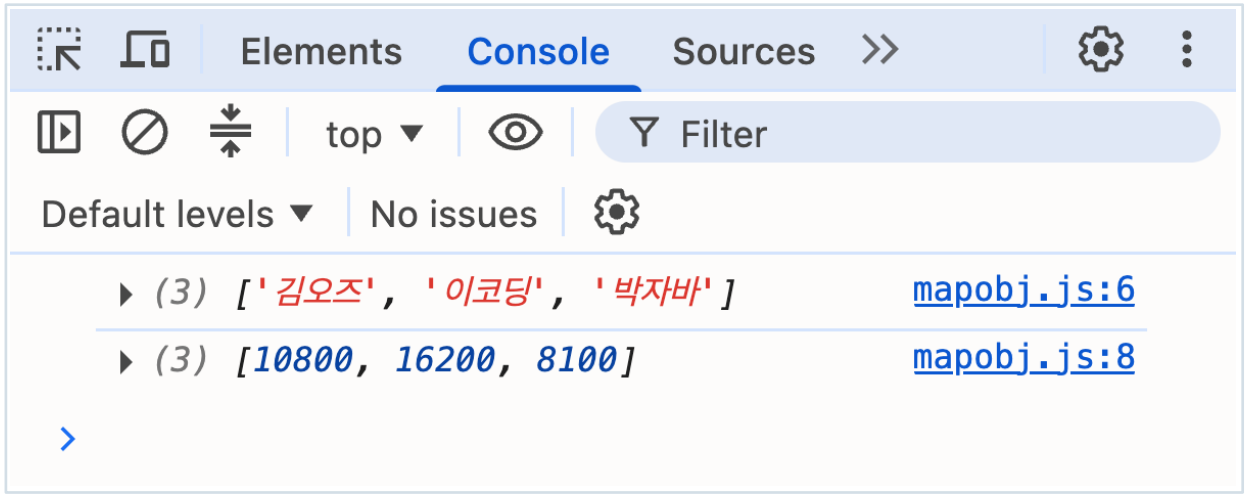
읽는 법

함수가 돌려준 값이 새 배열로
개수는 원래와 같은 3개
원래 numbers는 그대로



필요한 값만 꺼내거나 계산한 값 모으기

```
const users = [
  { name: "김오즈", age: 25 },
  { name: "이코딩", age: 30 },
  { name: "박자바", age: 20 },
];
console.log(users.map((user) => user.name));
const prices = [12000, 18000, 9000];
console.log(prices.map((price) => price * 0.9));
```

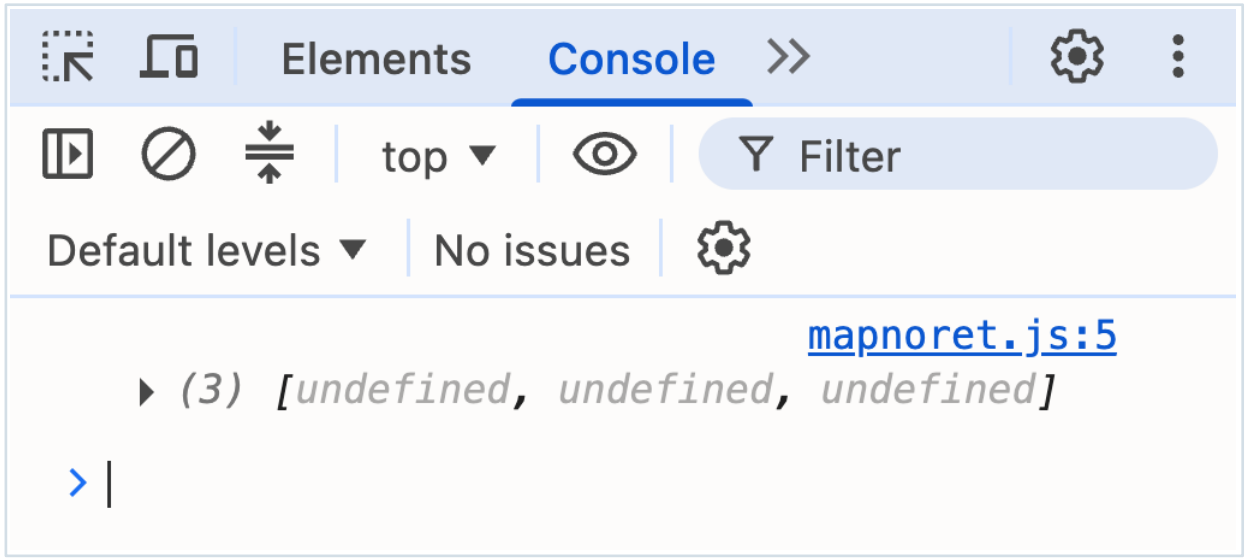


쓰는 곳

객체 배열 → 이름만 담은 배열
가격 배열 → 할인가 배열
화면에 보여 줄 값을 만들 때

return을 빠뜨리면 모이는 undefined

```
const numbers = [1, 2, 3];
const doubled = numbers.map((num) => {
  num * 2;                    // return이 없습니다
});
console.log(doubled);
```

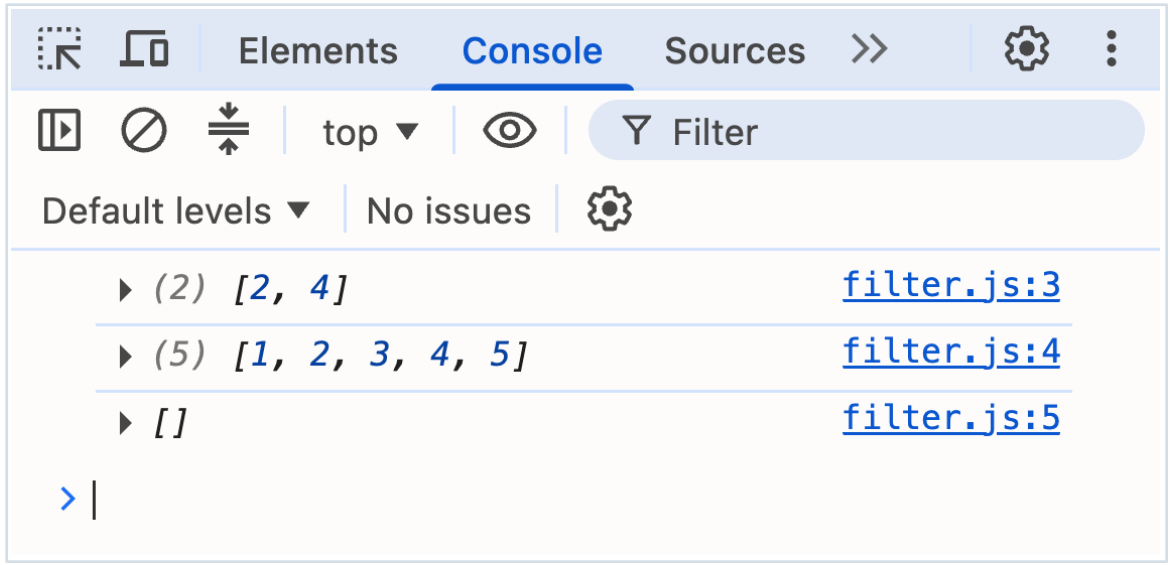


고치는 법

{ } 안이면 return num * 2;
한 줄이면 { }를 빼고
(num) => num * 2

조건이 true인 값만 새 배열에 담는 filter

```
const numbers = [1, 2, 3, 4, 5];
const evens = numbers.filter((num) => num % 2 === 0);
console.log(evens);
console.log(numbers);                          // 그대로
console.log(numbers.filter((num) => num > 10));    // 없으면
```

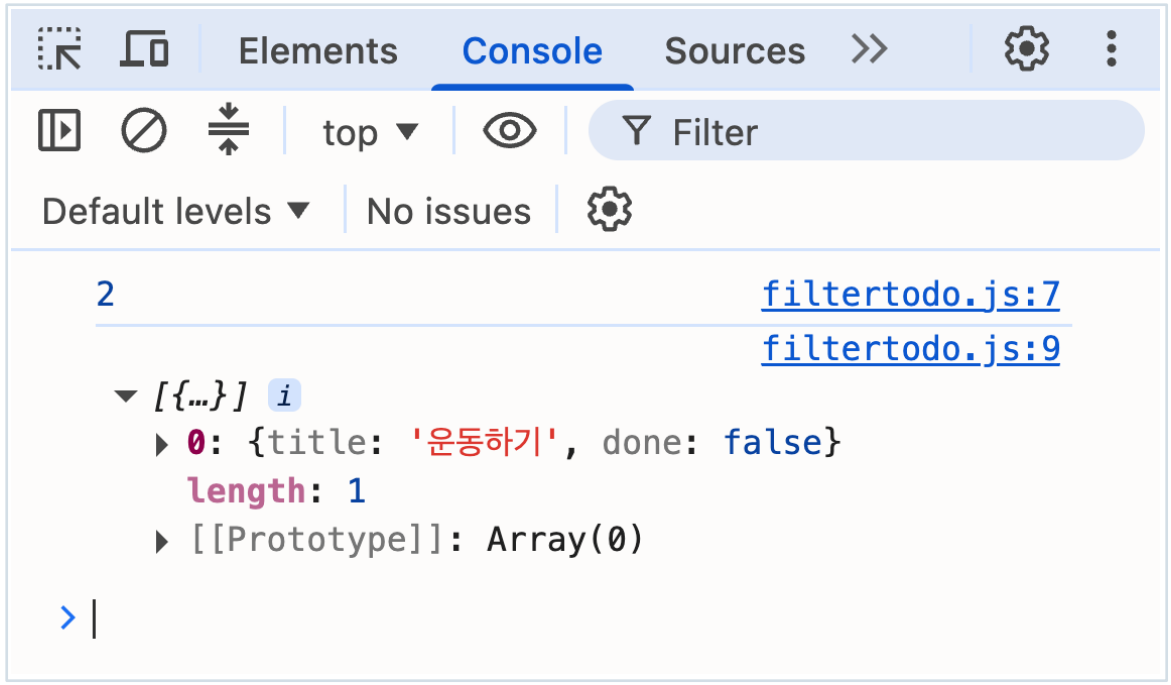


읽는 법

함수가 true를 돌려준 값만
개수가 줄 수 있습니다
하나도 없으면 빈 배열 []

끝낸 일과 남은 일 나누기

```
const todos = [
  { title: "장보기", done: true },
  { title: "운동하기", done: false },
  { title: "과제 제출", done: true },
];
const doneTodos = todos.filter((todo) => todo.done);
console.log(doneTodos.length);
const left = todos.filter((todo) => !todo.done);
console.log(left);
```



읽는 법

todo.done이 true면 남깁니다
!todo.done — 끝내지 않은 일
고른 객체가 통째로 남습니다

02 · 비교

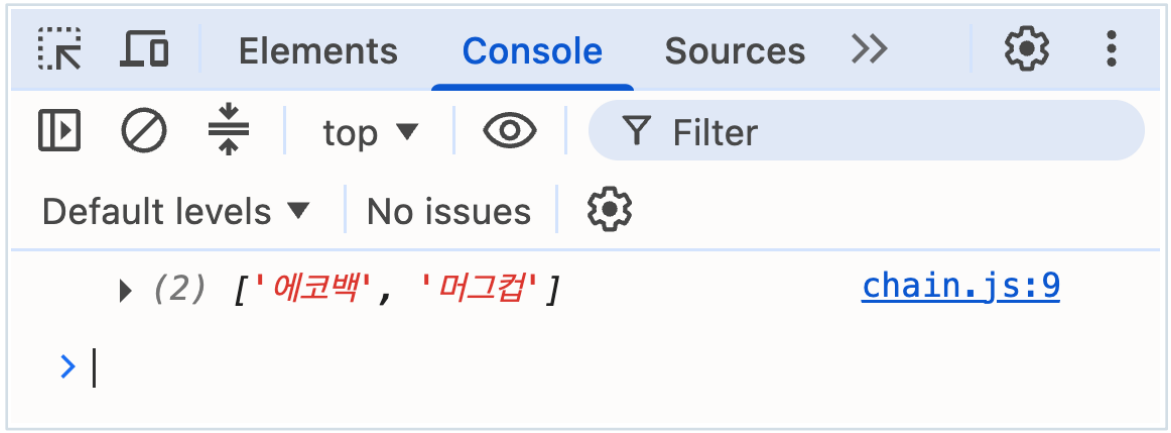
바꾸는 map, 고르는 filter

	`map`	`filter`
하는 일	값을 하나하나 바꾸기	조건에 맞는 값만 고르기
새 배열의 개수	원래와 같음	같거나 줄어듦 (없으면 [])
넘긴 함수가 돌려줄 것	새 배열에 담을 값	true 또는 false
원래 배열	그대로	그대로

바꾸려면 map, 고르려면 filter — 둘 다 새 배열을 돌려줍니다

고른 다음 바꾸기 — filter 뒤에 map

```
const products = [
  { id: 1, name: "에코백", price: 12000, stock: 5 },
  { id: 2, name: "텀블러", price: 18000, stock: 0 },
  { id: 3, name: "머그컵", price: 9000, stock: 12 },
];
const names = products
  .filter((p) => p.stock > 0)     // 재고 있는 상품
  .map((p) => p.name);            // 이름만
console.log(names);
```



읽는 순서

① filter — 재고가 있는 둘
② map — 그 둘의 이름
순서를 바꾸면 []가 됩니다

여기까지 정리

1	**map**	하나하나 바꾼 새 배열 — 개수는 그대로
2	**filter**	true인 값만 담은 새 배열 — 없으면 []
3	**return**	{ }를 쓰면 return 필수 — 빠뜨리면 undefined가 모입니다
4	**이어 쓰기**	filter로 고르고 map으로 바꾸기 — 점(.)으로 잇습니다

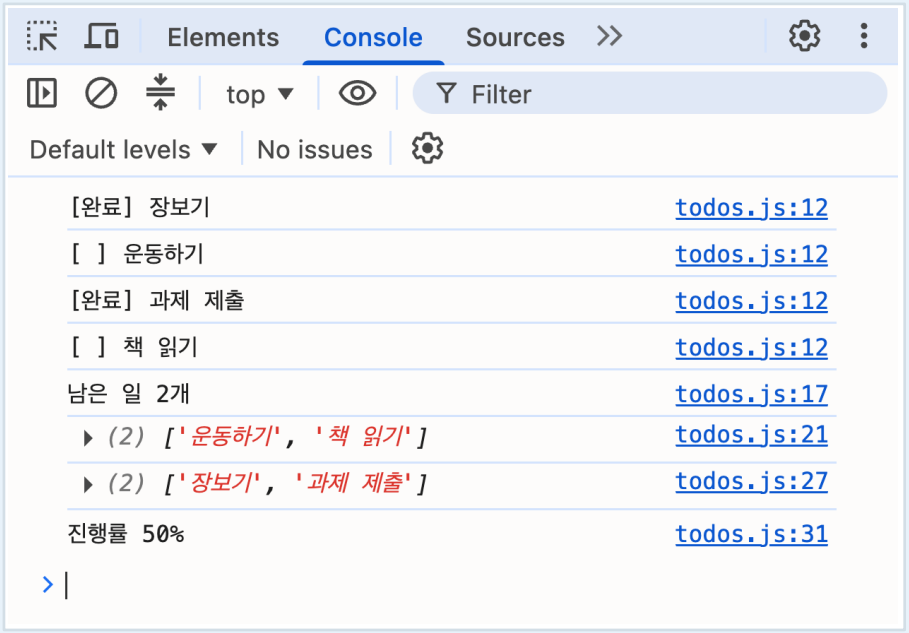
다음은 할 일 목록으로 forEach · filter · map을 연습합니다

실습 01

할 일 목록 다루기

todos.js에 할 일 네 개를 객체 배열로 만들고, 출력 · 고르기 · 제목 모으기 · 진행률을 구합니다.
forEach · filter · map과 이어 쓰기를 한 파일에서 모두 씁니다.

```
const todos = [ { id: 1, title: "장보기", done: true }, … ];
todos.forEach((todo) => {                    // 1. 전체 출력
  const mark = todo.done ? "[완료]" : "[ ]";
  console.log(`${mark} ${todo.title}`);
});
const left = todos.filter((todo) => !todo.done);    // 2
const leftTitles = left.map((todo) => todo.title);   // 3
// 4. filter → map 이어 쓰기   5. 진행률
```



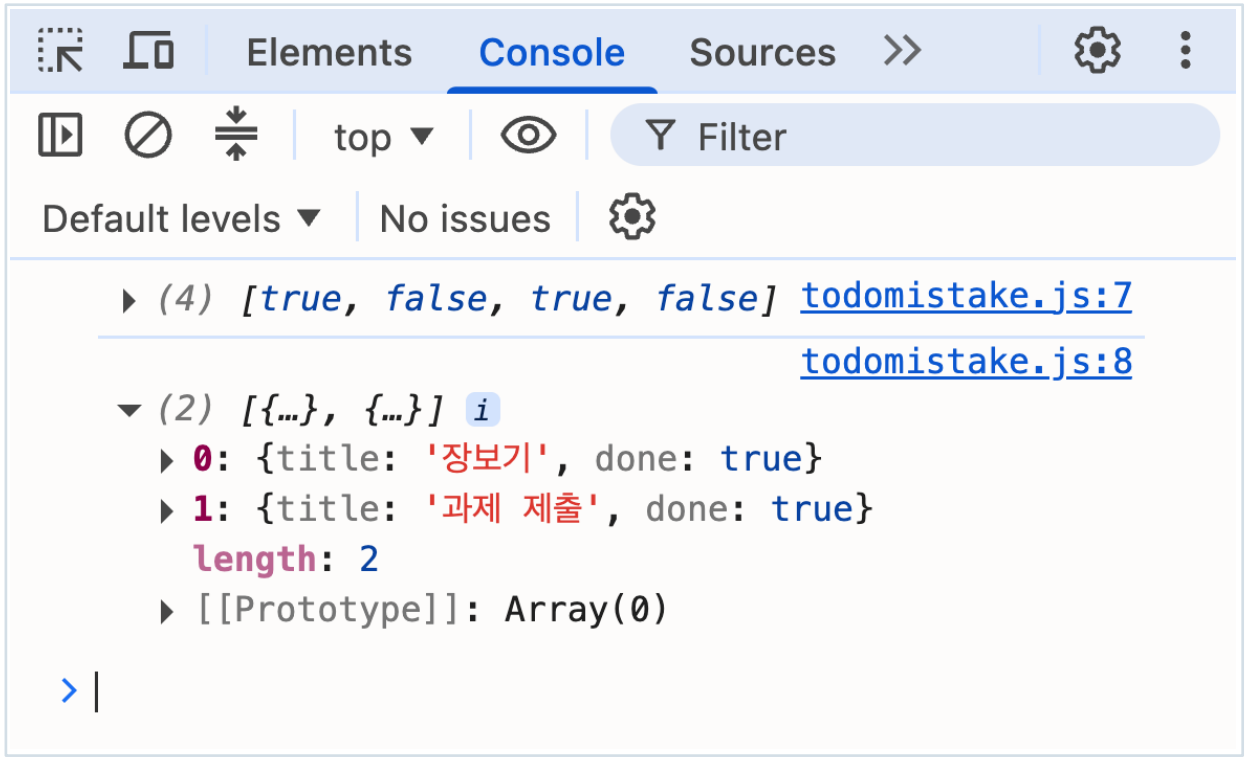
완성 파일의 실행 결과

고를 때 map을 쓰면 모이는 true · false

```
const todos = [
  { title: "장보기", done: true },
  { title: "운동하기", done: false },
  { title: "과제 제출", done: true },
  { title: "책 읽기", done: false },
];
console.log(todos.map((todo) => todo.done));
console.log(todos.filter((todo) => todo.done));
```

같은 함수, 다른 결과

map — 돌려준 true · false
filter — true인 객체만
고를 때는 filter입니다

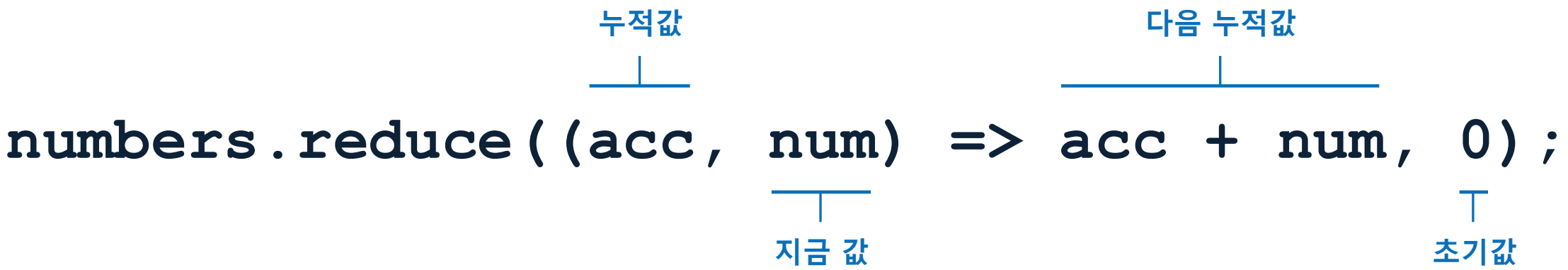


03

reduce와 찾기

reduce로 모으기 · 초기값 · find로 하나 찾기 · some · every로 확인하기

값을 하나씩 모아 값 하나로 만드는 reduce



함수가 돌려준 값이 다음 차례의 acc가 되고, 마지막 값이 결과입니다

맨 뒤의 0은 acc의 처음 값(초기값)입니다.

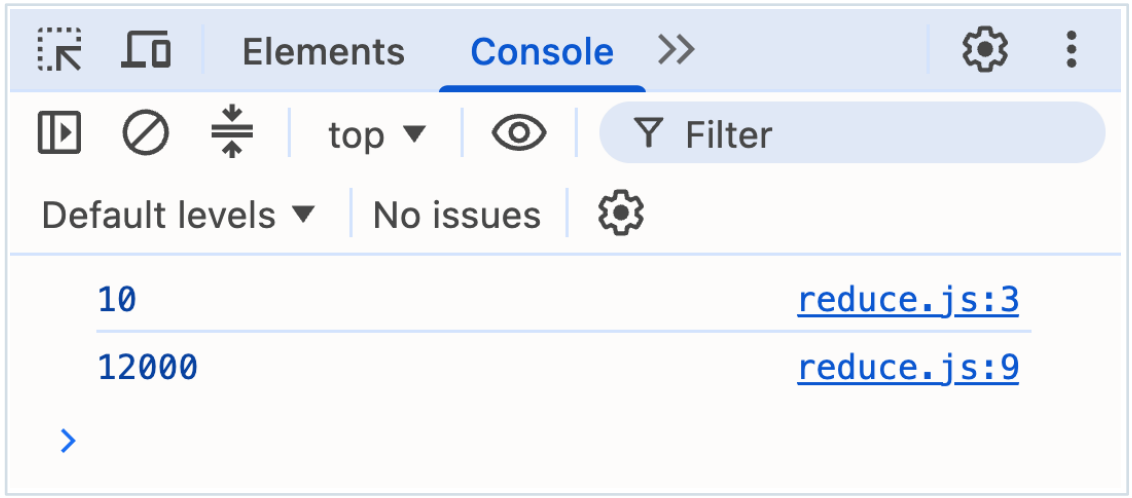
[1, 2, 3, 4]를 더하면 네 번 불리는 함수

차례	acc	num	돌려준 값 (acc + num)
1	0 ← 초기값	1	1
2	1	2	3
3	3	3	6
4	6	4	10 ← 결과

돌려준 값이 다음 차례의 acc가 됩니다 — 마지막 10이 reduce의 결과

합계와 장바구니 총액

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((acc, num) => acc + num, 0);
console.log(sum);
const cart = [
  { name: "책", price: 10000 },
  { name: "펜", price: 2000 },
];
const total = cart.reduce((acc, item) => acc + item.price, 0);
console.log(total);
```



읽는 법

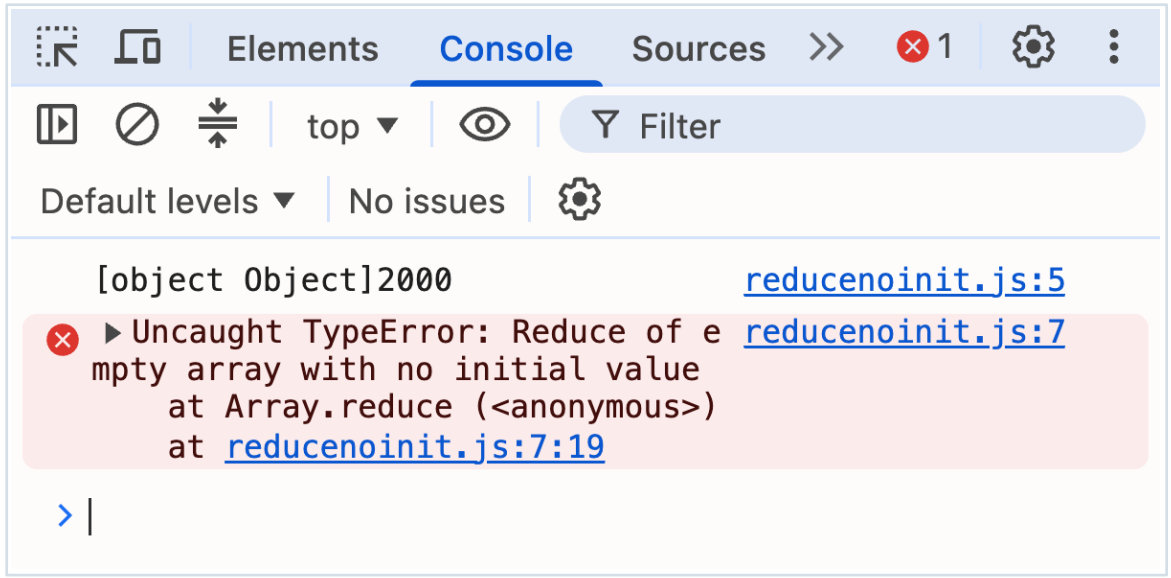
숫자 배열 — acc + num
객체 배열 — acc + item.price
초기값 0부터 시작합니다

초기값을 빼면 acc가 되는 첫 값

```
const cart = [
  { name: "책", price: 10000 },
  { name: "펜", price: 2000 },
];
console.log(cart.reduce((acc, item) => acc + item.price));
const empty = [];
console.log(empty.reduce((acc, num) => acc + num));
```

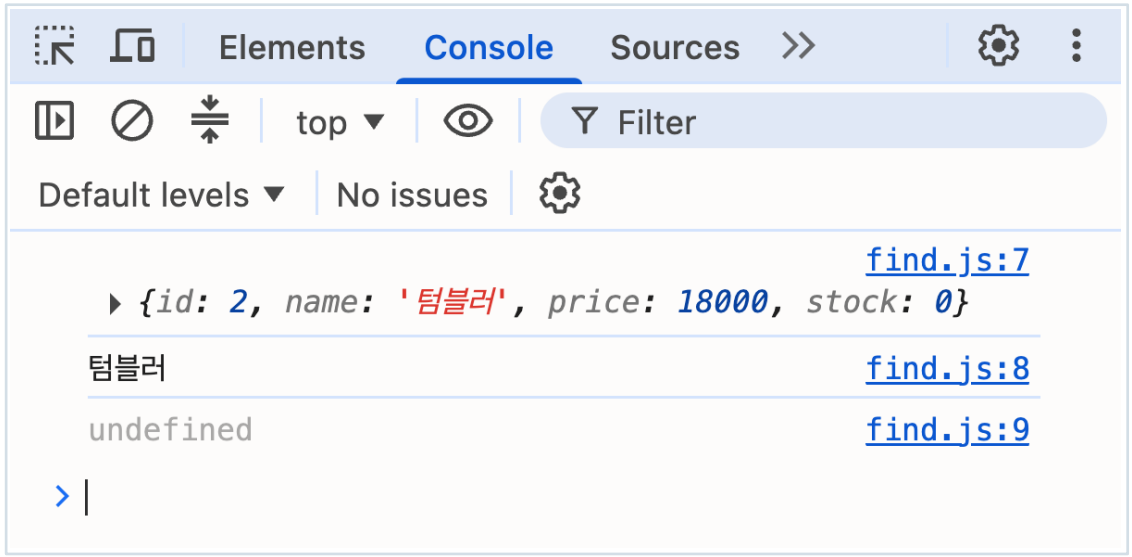
초기값은 늘 적기

객체 배열 — 첫 객체가 acc
빈 배열 — TypeError
합계는 끝에 , 0 을 붙입니다



조건에 맞는 첫 값 하나를 돌려주는 find

```
const products = [
  { id: 1, name: "에코백", price: 12000, stock: 5 },
  { id: 2, name: "텀블러", price: 18000, stock: 0 },
  { id: 3, name: "머그컵", price: 9000, stock: 12 },
];
const found = products.find((p) => p.id === 2);
console.log(found);
console.log(found.name);
console.log(products.find((p) => p.id === 9));   // 없으면
```



읽는 법

찾으면 그 객체 하나
못 찾으면 undefined
첫 값을 찾으면 멈춥니다

03 · 비교

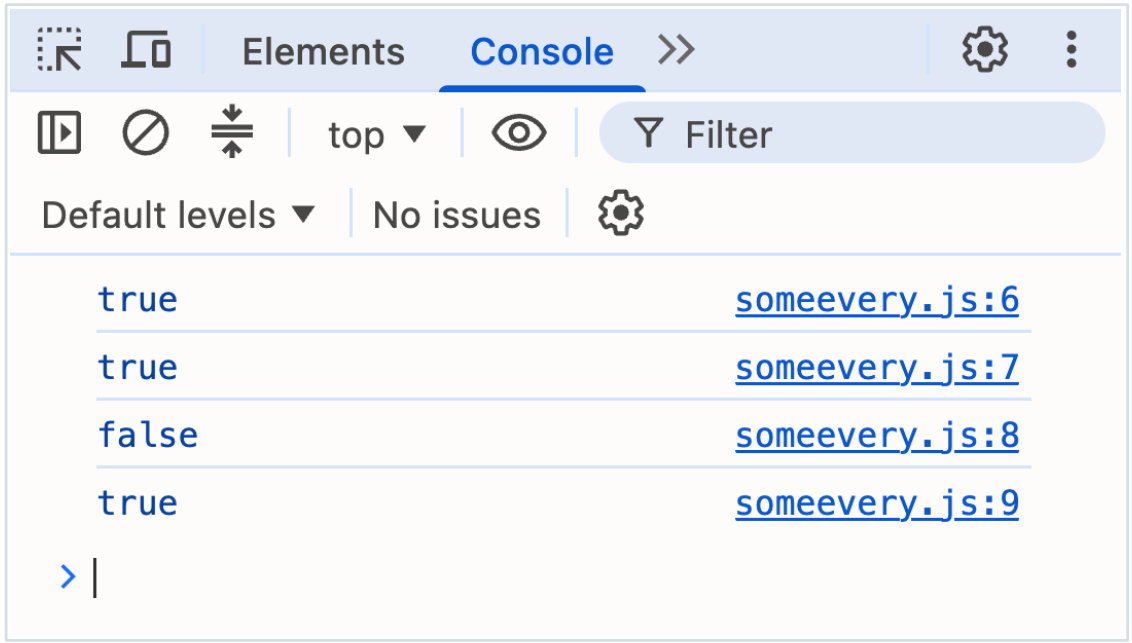
하나를 찾으면 find, 모두 고르면 filter

	`find`	`filter`
돌려주는 값	조건에 맞는 첫 값 하나	조건에 맞는 값 모두를 담은 배열
못 찾으면	`undefined`	`[]`
멈추는 때	처음 찾았을 때	끝까지 다 봅니다
쓰는 곳	id로 상품 하나 찾기	조건에 맞는 목록 만들기

find의 결과는 값 하나, filter의 결과는 배열입니다

하나라도 맞는가는 some, 모두 맞는가는 every

```
const products = [
  { id: 1, name: "에코백", price: 12000, stock: 5 },
  { id: 2, name: "텀블러", price: 18000, stock: 0 },
  { id: 3, name: "머그컵", price: 9000, stock: 12 },
];
console.log(products.some((p) => p.stock === 0));  // 품절이 있나
console.log(products.every((p) => p.price > 0));   // 모두 0원 초과
console.log(products.every((p) => p.stock > 0));   // 모두 재고
console.log([].every((n) => n > 0));               // 빈 배열
```



읽는 법

some — 하나라도 true면 true
every — 하나라도 false면 false
빈 배열이면 every는 true

03 · 정리

메서드마다 넘길 함수와 돌려받는 값

메서드	하는 일	넘긴 함수가 돌려줄 것	메서드가 돌려주는 값
`forEach`	값마다 실행	없음	`undefined`
`map`	하나하나 바꾸기	새 값	같은 개수의 새 배열
`filter`	고르기	true · false	고른 값의 새 배열
`reduce`	모으기	다음 누적값	마지막 누적값
`find`	하나 찾기	true · false	첫 값 또는 undefined
`some · every`	확인하기	true · false	`true / false`

다음은 원래 배열이 바뀌는지 따져 보고, 장바구니를 계산합니다

04

원본 배열과 데이터 가공

원래 배열이 바뀌는가 · 객체는 함께 바뀐다 · 할인 계산 · 장바구니 총액 · 설명 확인하기 · 실습 02

04 · 원본

원래 배열을 바꾸는 메서드

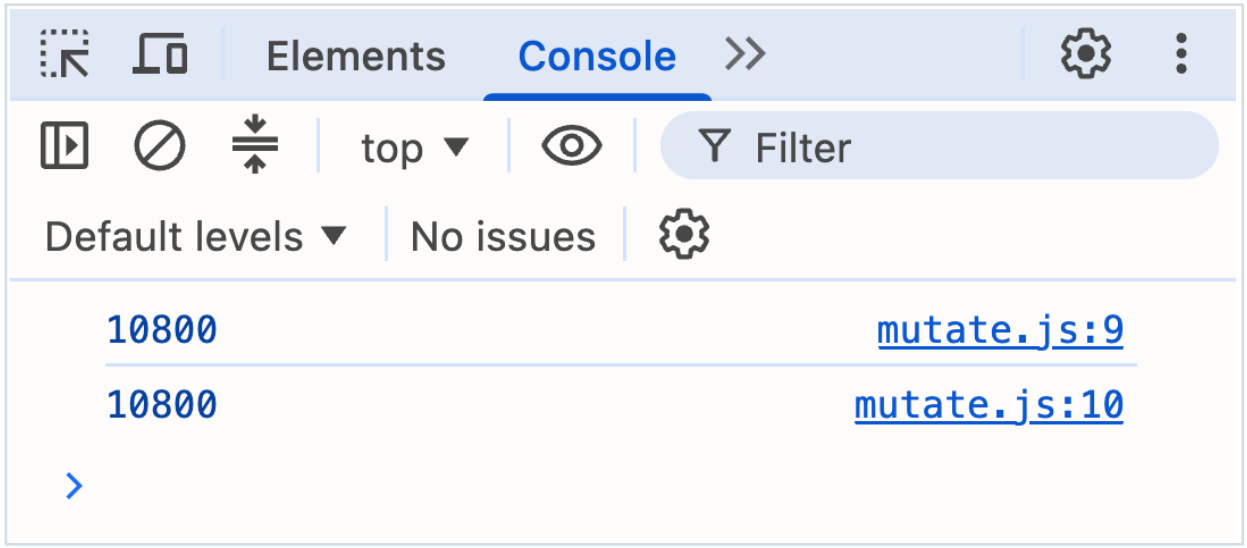
구분	메서드	돌려주는 것
원래 배열을 바꿈	`push · pop · shift · unshift · splice`	새 length · 뺀 값 · 지운 값들의 배열
	`sort · reverse`	바뀐 그 배열
새 배열을 돌려줌	`map · filter · slice`	새 배열 (원래는 그대로)
값 하나를 돌려줌	`reduce · find · some · every`	모은 값 · 찾은 값 · true / false
돌려주는 값 없음	`forEach`	`undefined`

오늘 배운 메서드는 원래 배열을 바꾸지 않습니다

단, 넘긴 함수 안에서 배열 속 객체를 직접 고치면 원래 객체도 바뀝니다 — 다음 장.

map 안에서 고치면 함께 바뀌는 원래 객체

```
const products = [
  { name: "에코백", price: 12000 },
  { name: "머그컵", price: 9000 },
];
const sale = products.map((p) => {
  p.price = p.price * 0.9;    // 원래 객체를 고침
  return p;
});
console.log(sale[0].price);
console.log(products[0].price);
```

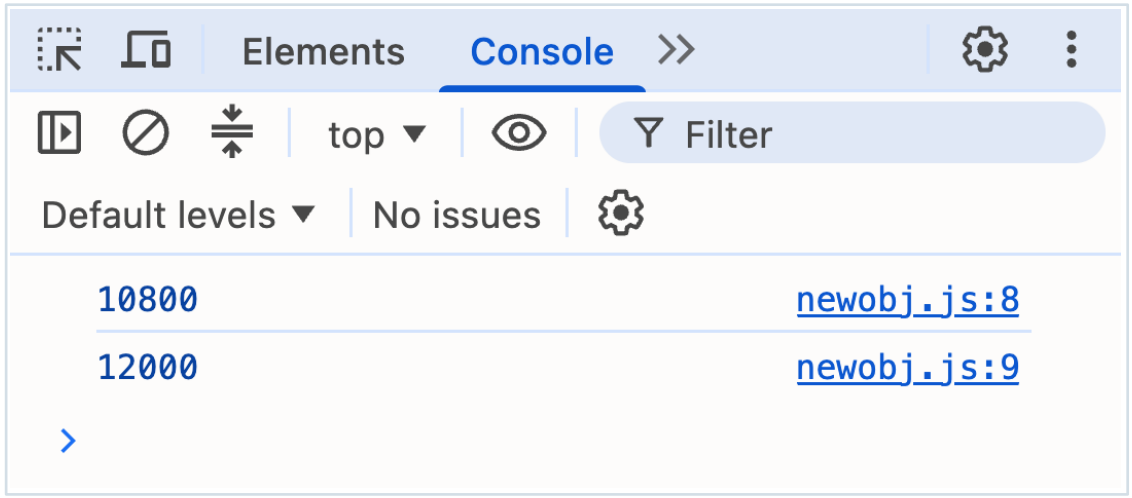


읽는 법

p는 원래 객체 그 자체
고치면 products도 바뀝니다
원래 가격 12000이 사라짐

새 객체를 만들어 돌려주면 남는 원래 값

```
const products = [
  { name: "에코백", price: 12000 },
  { name: "머그컵", price: 9000 },
];
const sale = products.map((p) => {
  return { name: p.name, price: p.price * 0.9 };  // 새 객체
});
console.log(sale[0].price);
console.log(products[0].price);
```

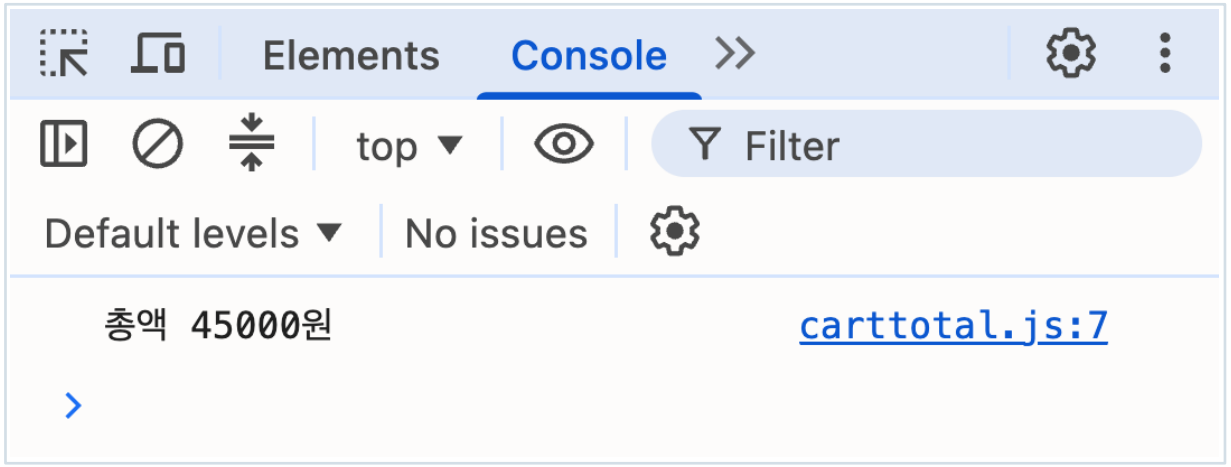


읽는 법

{ }로 새 객체를 만들어 돌려줌
sale은 할인가, products는 원래 값
원래 데이터를 지키는 방법

장바구니 총액 — 가격 × 수량 모으기

```
const cart = [
  { name: "에코백", price: 12000, qty: 1 },
  { name: "머그컵", price: 9000, qty: 2 },
  { name: "키링", price: 5000, qty: 3 },
];
const total = cart.reduce((sum, item) => sum + item.price * item.qty, 0);
console.log(`총액 ${total}원`);
```

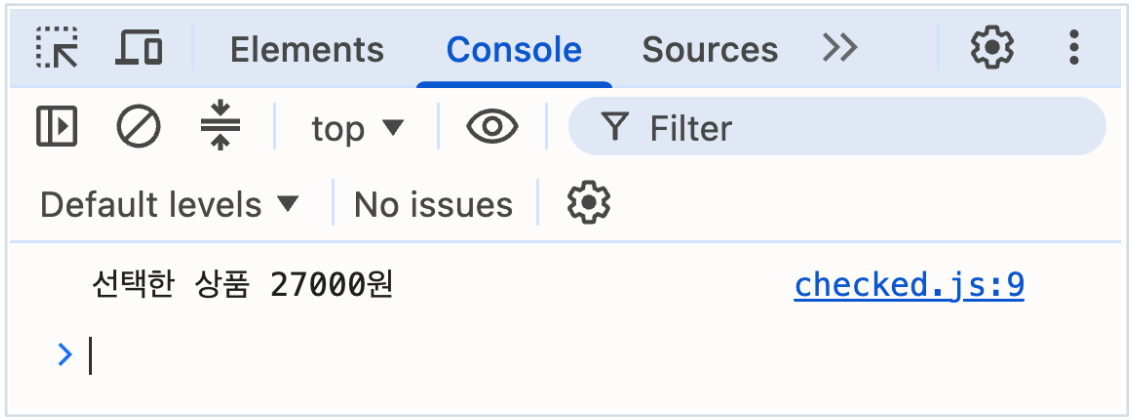


계산

12000 × 1 + 9000 × 2 + 5000 × 3
= 45000원
sum의 초기값 0부터 더합니다

선택한 상품만 골라 총액 내기

```
const cart = [
  { name: "에코백", price: 12000, qty: 1, checked: true },
  { name: "머그컵", price: 9000, qty: 2, checked: false },
  { name: "키링", price: 5000, qty: 3, checked: true },
];
const total = cart
  .filter((item) => item.checked)
  .reduce((sum, item) => sum + item.price * item.qty, 0);
console.log(`선택한 상품 ${total}원`);
```

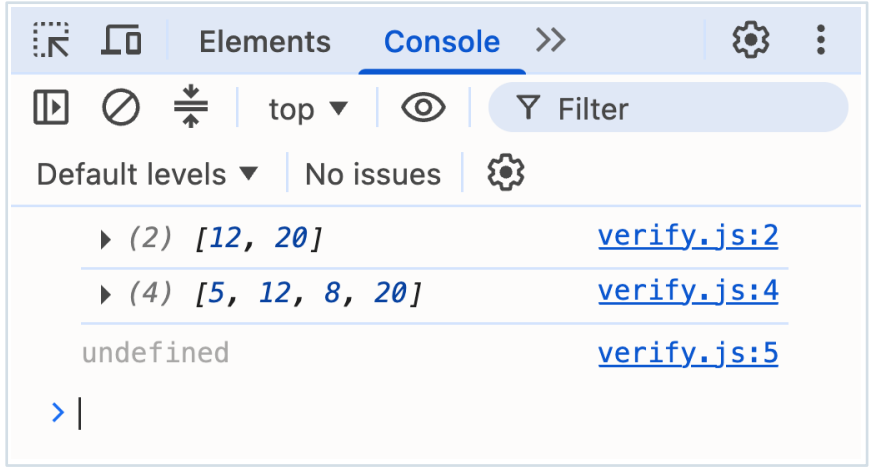


읽는 순서

① filter — 선택한 두 상품
② reduce — 가격 × 수량의 합
머그컵 18000원은 빠집니다

설명이 맞는지 Console로 확인

```
const nums = [5, 12, 8, 20];
console.log(nums.filter((n) => n > 10));
const tens = nums.map((n) => n * 10);
console.log(nums);
console.log(nums.find((n) => n > 100));
```



들은 설명	실제로 실행하면
① filter는 조건에 맞는 첫 값 하나를 돌려준다	[12, 20] — 맞는 값을 모두 담은 배열
② map은 원래 배열을 바꾼다	nums는 [5, 12, 8, 20] 그대로
③ find는 못 찾으면 빈 배열 []을 돌려준다	undefined

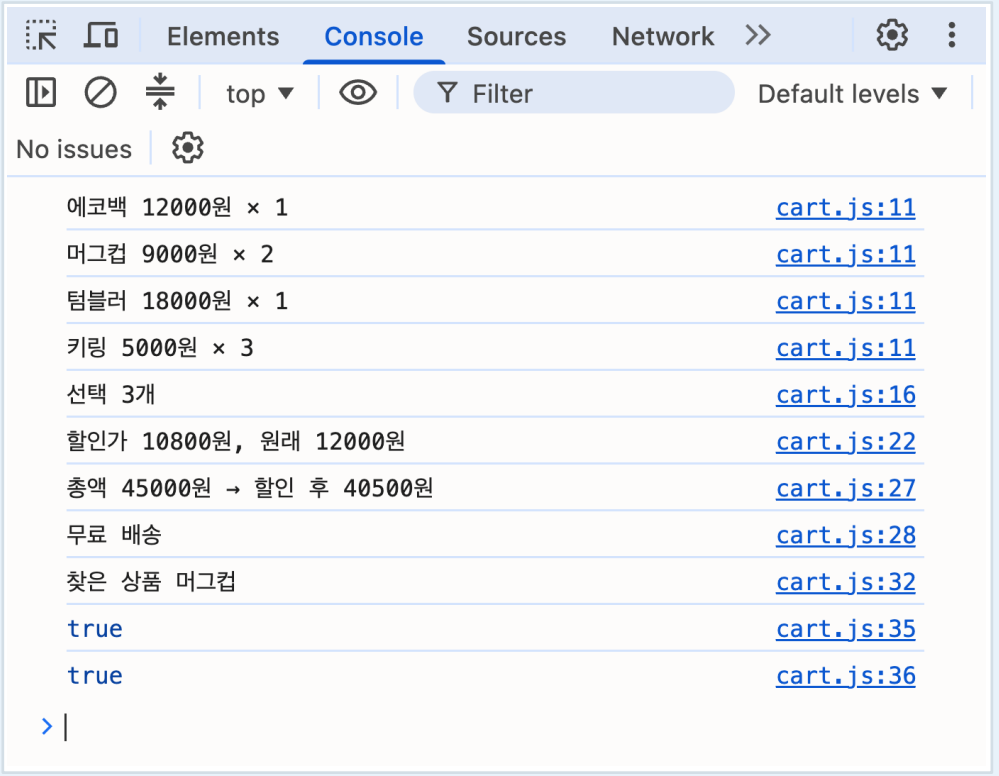
실습 02

장바구니 계산하기

cart.js에 상품 네 개를 담고, 선택 · 할인 · 총액 · 찾기 · 확인을 차례로 합니다.
오늘 배운 메서드 일곱 개를 한 파일에서 모두 씁니다.

만들 것

① forEach로 담은 상품 출력
② filter로 선택한 상품만 고르기
③ map으로 10% 할인한 새 객체 만들기
④ reduce로 할인 전 · 후 총액, 무료 배송 확인
⑤ find로 id가 2인 상품 찾기
⑥ some · every로 수량과 가격 확인하기



실습 02

일부러 틀려 보기

완성한 cart.js를 한 곳씩 바꿔 저장하고, 결과가 어떻게 바뀌는지 본 뒤 되돌립니다.
첫 번째만 빨간 줄이 뜨지 않습니다. 결과를 읽고 원인을 찾습니다.

바꿀 곳	결과
① 4단계 첫 reduce의 , 0 을 지우기	총액 [object Object]1800015000원 → 할인 후 40500원 — 빨간 줄 없음
② 3단계의 selected.map을 selected.forEach로	TypeError: Cannot read properties of undefined (reading '0')
③ 5단계의 item.id === 2 를 item.id === 9 로	TypeError: Cannot read properties of undefined (reading 'name')

세 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 02 끝

점검

배열 메서드에서 자주 나오는 세 가지

!

undefined가 모였어요

{ } 안에 return을
빠뜨렸습니다. 한 줄이면
{ }를 빼도 됩니다.

!

[object Object]...

reduce의 초기값을
빠뜨렸습니다.
끝에 , 0 을 붙입니다.

!

undefined에서 TypeError

find가 못 찾았거나
forEach의 결과를 담았습니다.
한 칸 앞을 찍어 봅니다.

넘긴 함수가 무엇을 돌려주는지부터 봅니다

map은 새 값, filter · find · some · every는 true · false, reduce는 다음 누적값입니다.

WRAP UP

오늘 정리

개념

메서드에 넘기는 함수가 콜백

forEach는 실행만, 돌려주는 값 없음

map은 바꾸고, filter는 고른다

reduce는 모아서 값 하나

find는 하나, some · every는 true · false

오늘 배운 메서드는 원래 배열을 바꾸지 않는다

직접 쓸 수 있어야 하는 것

forEach로 하나씩 출력하기

map · filter로 새 배열 만들기

filter → map 이어 쓰기

reduce로 합계 · 총액 내기

find · some · every로 찾고 확인하기

새 객체를 만들어 원래 값 지키기

참고교재

교재에서 함께 읽을 곳

오늘 수업과 이어지는 장입니다. 복습할 때 장 번호로 찾아 읽습니다.

길벗캠퍼스

『백엔드 입문자를 위한 모던 자바스크립트 & Node.js』

이창현 지음

8장. 사건해결, 이벤트와 이벤트 리스너
9장. ES2015+ 문법

위키북스

『모던 자바스크립트 Deep Dive』

이웅모 지음

27장. 배열
28장. Number
29장. Math
30장. Date
32장. String
35장. 스프레드 문법

NEXT

DAY 15

DOM 조작 기초

JavaScript로 HTML 요소를 고르고 바꾸고,
배열 데이터로 카드 목록을 만들어
화면에 직접 넣습니다.

수고하셨습니다.